Deployment — A Careful Script With No Rollback

InstantMudra Enterprises Admin Portal

deploy.bat → deploy.ps1 — the deployment pipeline, step by step

1. Check SSH connectivity
abort if VPN/server unreachable

2. git fetch + detached worktree
clean build from origin/master, not local state

3. mvnw clean install -DskipTests
tests never gate a deployment

4. scp WAR to server (/tmp)
dedicated key, batch mode, no password fallback

— remote, over SSH, as a single base64-encoded bash script —

5. sudo -n mkdir backups; cp current WAR → timestamped backup file

6. shutdown.sh, then poll up to 150s for Tomcat to actually stop
aborts deployment (exit 1) if Tomcat is still running — avoids a double instance

7. sudo -n rm -f DEPLOY_WAR ; sudo -n rm -rf EXPLODED_DIR
the OLD, currently-working WAR is deleted HERE — before the new one is confirmed to work
only copy of the working version left is the untouched backup file from step 5

8. mv new WAR into place; chmod 664; startup.sh

9. Confirm success by: process count > 0, AND exploded WAR directory appears within 60s
no HTTP health check — a running process with an exploded WAR can still have a Spring context that failed to start

⚠ CRITICAL — the backup from step 5 is never automatically restored

Both failure exits — "Tomcat failed to start" and "Application deployment folder was not created" — do the same thing:
`tail -n 100 catalina.out ; exit 1`

Neither branch copies $BACKUP_FILE back to $DEPLOY_WAR or restarts Tomcat with the known-good version.
A failed deployment leaves the site down, with the old working WAR sitting only in a backup file —
recovery requires someone to SSH in and restore it by hand.

Other findings

No CI/CD pipeline exists
Confirmed by absence: no .github/workflows, no pipeline YAML anywhere in the repo.
Deployment is fully manual — a developer runs deploy.bat locally, requiring VPN access and a locally-held SSH key.

Passwordless sudo on the deploy account
Every remote file operation uses sudo -n (non-interactive). A leaked deploy SSH key grants passwordless root-level file access on the production Tomcat server — no second factor stands between key and shell.

Single hardcoded target, no env selection
deploy-config.ps1 hardcodes one host (10.1.3.5) and one Tomcat path. This script has no visibility into which Spring profile (uat/prod, Module 1) the deployed WAR actually runs under on that server.

.gitignore confirms, firsthand, that Module 1's finding is real
No application*.properties pattern is excluded anywhere in .gitignore — the plaintext DB passwords, vendor API keys, and AES base secret documented in Module 1 are genuinely committed to source control, not just locally present and untracked.

By contrast: this is a carefully engineered script in several real respects

- Reproducible builds — a detached git worktree from origin/master, not whatever happens to be on the local disk
- Dedicated SSH key, IdentitiesOnly + BatchMode (no interactive password fallback), sane connection timeouts
- Waits for Tomcat to actually stop before touching files, and aborts outright rather than risking a double-instance
- Waits for the exploded WAR directory post-restart, and tails the last 100 log lines on any failure for diagnostics
- Cleans up the temporary git worktree on both the success and failure paths
- Base64-encodes the remote bash script specifically to avoid CRLF/line-ending corruption over SSH — a real, subtle problem, deliberately solved

Sound engineering, by contrast | Standard step / minor finding | Flagged risk | Critical — confirmed operational gap